Overview of Retrieval augmented generation

Subject: Retrieval augmented generation

1.

Applications Retrieval-augmented generation is used in applications where generated responses need to be grounded in external or frequently updated information. Commonly cited use cases include search engines, question-answering systems, customer support chatbots, enterprise knowledge assistants, content generation, recommendation systems, retail and e-commerce, and industrial or manufacturing workflows. In healthcare, RAG has been studied as a way to ground large language model outputs in external medical knowledge sources, although reviews have noted continuing challenges around evaluation, ethics, and clinical reliability.

2.

RAG and LLM limitations LLMs can provide incorrect information. For example, when Google first demonstrated its LLM tool "Google Bard" (later re-branded to Gemini), the LLM provided incorrect information about the James Webb Space Telescope. This error contributed to a $100 billion decline in Google's stock value. RAG is used to prevent these errors, but it does not solve all the problems. For example, LLMs can generate misinformation even when pulling from factually correct sources if they misinterpret the context. MIT Technology Review gives the example of an AI-generated response stating, "The United States has had one Muslim president, Barack Hussein Obama." The model retrieved this from an academic book rhetorically titled Barack Hussein Obama: America's First Muslim President? The LLM did not "know" or "understand" the context of the title, generating a false statement. LLMs with RAG are programmed to prioritize new information. This technique has been called "prompt stuffing." Without prompt stuffing, the LLM's input is generated by a user; with prompt stuffing, additional relevant context is added to this input to guide the model's response. This approach provides the LLM with key information early in the prompt, encouraging it to prioritize the supplied data over pre-existing training knowledge.

3.

Encoder These methods focus on the encoding of text as either dense or sparse vectors. Sparse vectors, which encode the identity of a word, are typically dictionary-length and contain mostly zeros. Dense vectors, which encode meaning, are more compact and contain fewer zeros. Various enhancements can improve the way similarities are calculated in the vector stores (databases).

4.

Performance improves by optimizing how vector similarities are calculated. Dot products enhance similarity scoring, while approximate nearest neighbor (ANN) searches improve retrieval efficiency over K-nearest neighbors (KNN) searches. Accuracy may be improved with Late Interactions, which allow the system to compare words more precisely after retrieval. This helps refine document ranking and improve search relevance. Hybrid vector approaches may be used to combine dense vector representations with sparse one-hot vectors, taking advantage of the computational efficiency of sparse dot products over dense vector operations. Other retrieval techniques focus on improving accuracy by refining how documents are selected. Some retrieval methods combine sparse representations, such as

SPLADE, with query expansion strategies to improve search accuracy and recall.

5.

Challenges RAG does not prevent hallucinations in LLMs. According to Ars Technica, "It is not a direct solution because the LLM can still hallucinate around the source material in its response." While RAG improves the accuracy of large language models (LLMs), it does not eliminate all challenges. One limitation is that while RAG reduces the need for frequent model retraining, it does not remove it entirely. Additionally, LLMs may struggle to recognize when they lack sufficient information to provide a reliable response. Without specific training, models may generate answers even when they should indicate uncertainty. According to IBM, this issue can arise when the model lacks the ability to assess its own knowledge limitations.

6.

Evaluation and benchmarks RAG systems are commonly evaluated using benchmarks designed to test retrievability, retrieval accuracy and generative quality. Popular evaluation resources include BEIR, a heterogeneous benchmark for zero-shot information retrieval across multiple datasets, and Natural Questions, a large-scale question answering dataset released by Google Research.

7.

Process Retrieval-augmented generation (RAG) enhances large language models (LLMs) by incorporating an information-retrieval mechanism that allows models to access and utilize additional data beyond their original training set. Ars Technica notes that "when new information becomes available, rather than having to retrain the model, all that’s needed is to augment the model’s external knowledge base with the updated information" ("augmentation"). IBM states that "in the generative phase, the LLM draws from the augmented prompt and its internal representation of its training data to synthesize" an answer.

8.

Hybrid search Sometimes vector database searches can miss key facts needed to answer a user's question. One way to mitigate this is to do a traditional text search, add those results to the text chunks linked to the retrieved vectors from the vector search, and feed the combined hybrid text into the language model for generation. The adoption of RAG in consumer-facing web search products has given rise to new content optimization disciplines, as practitioners have noted that content retrievability in RAG systems depends on factors like semantic structure, passage-level authority signals, and entity clarity rather than traditional search ranking signals such as backlinks.

9.

Fixed length with overlap. This is fast and easy. Overlapping consecutive chunks helps to maintain semantic context across chunks. Syntax-based chunks can break the document up into sentences. Libraries such as spaCy or NLTK can also help. File format-based chunking. Certain file types have natural chunks built in, and it's best to respect them. For example, code files are best chunked and vectorized as whole functions or classes. HTML files should leave <table> or base64 encoded <img> elements intact. Similar considerations should be taken for pdf files. Libraries such as Unstructured or LangChain can assist with this method.

10.

Pre-training the retriever using the Inverse Cloze Task (ICT), a technique that helps the model learn retrieval patterns by predicting masked text within documents. Supervised retriever optimization aligns retrieval probabilities with the generator model's likelihood distribution. This involves retrieving the top-k vectors for a given prompt, scoring the generated response's perplexity, and minimizing KL divergence between the retriever's selections and the model's likelihoods to refine retrieval. Reranking techniques can refine retriever performance by prioritizing the most relevant retrieved documents during training.

11.

RAG poisoning RAG systems may retrieve factually correct but misleading sources, leading to errors in interpretation. In some cases, an LLM may extract statements from a source without considering its context, resulting in an incorrect conclusion. Additionally, when faced with conflicting information, RAG models may struggle to determine which source is accurate. The worst case outcome of this limitation is that the model may combine details from multiple sources producing responses that merge outdated and updated information in a misleading manner. According to the MIT Technology Review, these issues occur because RAG systems may misinterpret the data they retrieve.